Frequency De-Mixer: Unwanted Solo

EE200 Practical Assignment Q2 — Mukund Advaith Balaji

July 5, 2025

Abstract

This practical assignment explores the concept of a frequency de-mixer. We aim to analyse an audio file containing primarily low frequency indie music and high frequency piccolo music, and remove the unwanted piccolo music, producing a cleaned audio file only containing the indie music.

1 Introduction

In sound engineering, engineers come across many cases where different frequency components of audio needs to be separated, for purposes such as removing noise, amplifying certain parts of audio etc.

In this case, we aim to remove certain parts (unwanted instrumental) of an audio file containing music, in order to restore the harmony of the music.

We will use Python and a series of its libraries to manipulate and edit the audio, as well as export it for our use after editing.

2 Continuous Fourier Transform

Given a function x(t) is a finite continuous signal. To analyze the frequencies contained within the signal, we need to find Continuous Fourier Transform of X(jω). The Continuous Fourier Transform is given by:

$$X(j\omega) = \int_{-\infty}^{\infty} x(t) \cdot e^{-j\omega t}\, dt$$

And the Inverse 2D Discrete Fourier Transform is given by:

$$x(t) = \frac{1}{2\pi} \int_{-\infty}^{\infty} X(j\omega) \cdot e^{j\omega t}\, d\omega$$

We use the Fast Fourier Transform Algorithm for this purpose, while programming.

3 Spectrogram and Short-Time Fourier Transform

A Spectrogram is a visual representation of the Short-Time Fourier Transform (STFT) of a signal. The STFT breaks the signal into short overlapping segments, applies a window function to each segment, and then computes the Fourier transform of each windowed segment. This gives a time-frequency representation. The Short-Time Fourier Transform is given by:

$$\mathbf{STFT}\{x(t)\}(\tau, \omega) \equiv X(\tau, \omega) = \int_{-\infty}^{\infty} x(t)\, w(t-\tau)\, e^{-j\omega t}\, dt$$

Where w(t) is an appropriate window function. The magnitude of STFT is plotted as a Spectrogram.

4 Power Spectral Density

The Power Spectral Density (PSD) of a signal describes how the power of the signal is distributed over different frequencies. We can roughly say that it is closely related to the square of the magnitude of Continuous Fourier Transform.

Mathematically, it is calculated by taking the Fourier transform of its autocorrelation function (the autocorrelation function measures the similarity of a signal with a time-delayed copy of itself), or by using the Welch's Method.

5 Tools for Implementation

We will be using Python to manipulate the audio, as well as the following libraries:

- NumPy: For Fourier transform, matrix and mathematical operations.
- Librosa and Librosa Display: For audio manipulation, such as loading, normalisation etc and displaying waveform, spectrogram etc, repectively.
- Matplotlib: For graphical visualisation and display.
- SciPy Signal: For other signal manipulation fucntions.

And some supplementary libraries such as:

- Soundfile: To create audio files.
- Audio (from IPython Display): For audio playing GUI in code output.

6 Implementation

- Loading of Audio into code: The audio is loaded into the code using the Librosa library. The sampling rate and duration of the audio is printed and the audio is normalised. The waveform of the audio is also plotted.
- Continuous Fourier Transform: Using the inbuilt function rfft from NumPy library, continuous Fourier Transform of audio is calculated. It is plotted using Matplotlib.

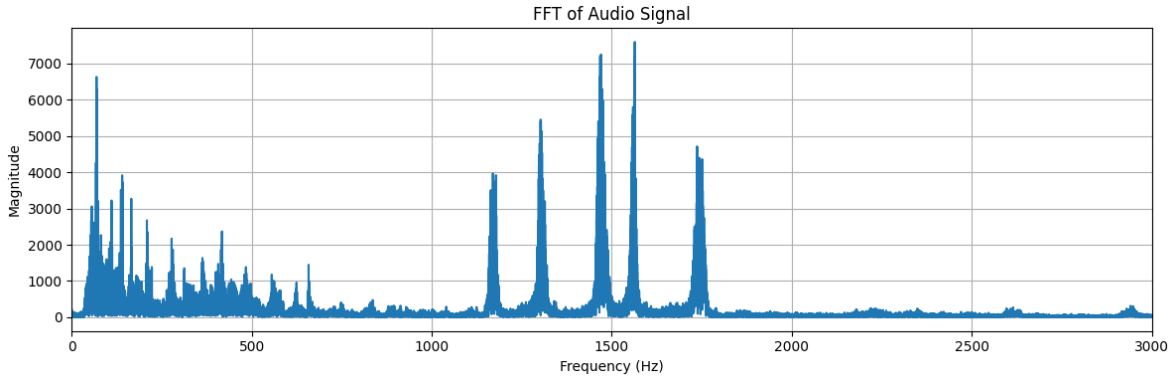

Figure 1: Fourier Transform of Audio

- Spectrogram: Using the inbuilt function stft from Librosa library, and Matplotlib, spectrogram is plotted.

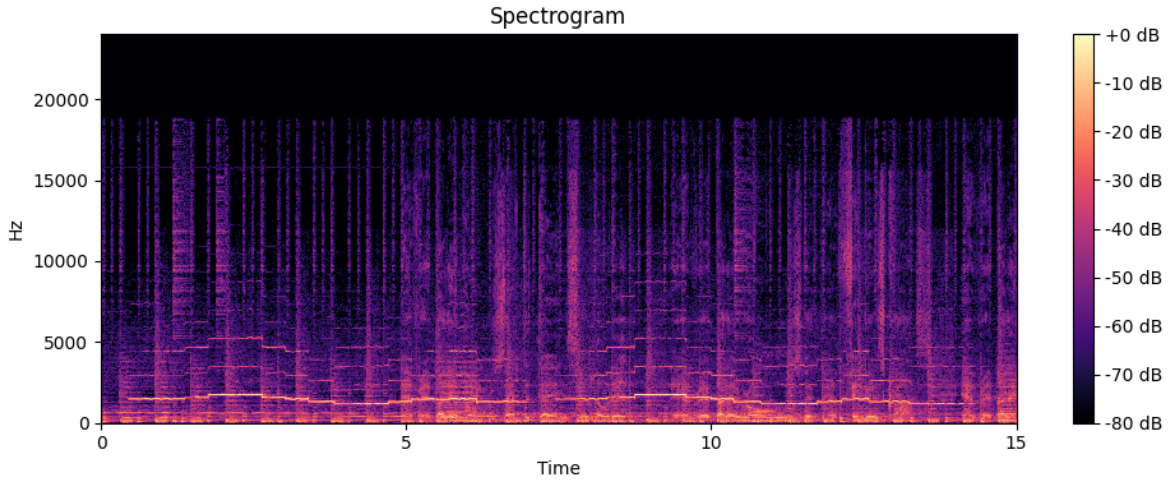

Figure 2: Spectrogram of Audio

- Power Spectral Density: Using the inbuilt function welch from Scipy Signal library, and Matplotlib, Power Spectral Density is plotted.

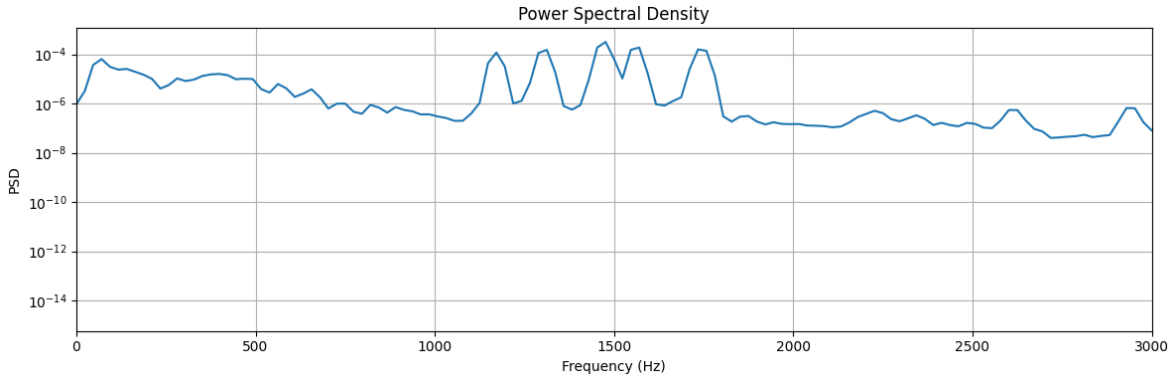

Figure 3: Power Spectral Density of Audio

7 Analysis and Filtering

From the Fourier Transform Plot (and PSD plot), we can see 5 out-of-place sharp peaks corresponding to high frequency components, after 1100 Hz, which likely belong to the piccolo. We can remove these using a low pass filter.

Low Pass Filter: It is a type of filter which allows all signals below a certain frequency, but restricts signals above this frequency.

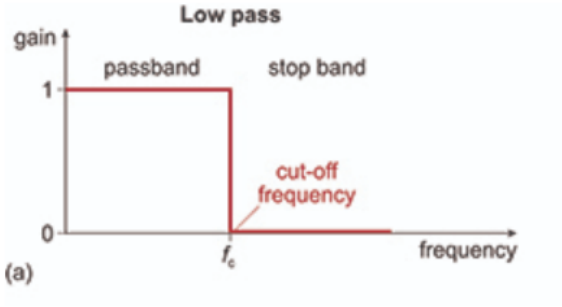

Figure 4: Low Pass Filter

In this assignment, it is implemented using NumPy array. The array of values is copied and all values above the cutoff frequency are set to zero. Then, inverse Fourier Transform is taken of this new array and normalized to get the audio.

Cons: There might be some small high frequency components in the indie song as well, which may be removed, like vocals.

8 Final Cleaned Audio Formation

After applying your choice of filters, the audio is saved using functions from the SoundFile library. The Fourier Transform, Spectrogram and Power Spectral Density of the cleaned audio is also plotted for comparison.

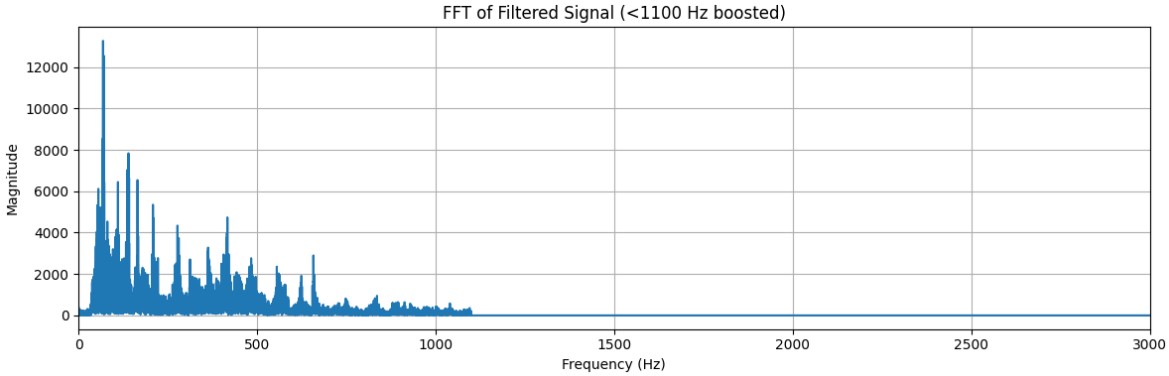

Figure 5: Fourier Transform of Cleaned Audio

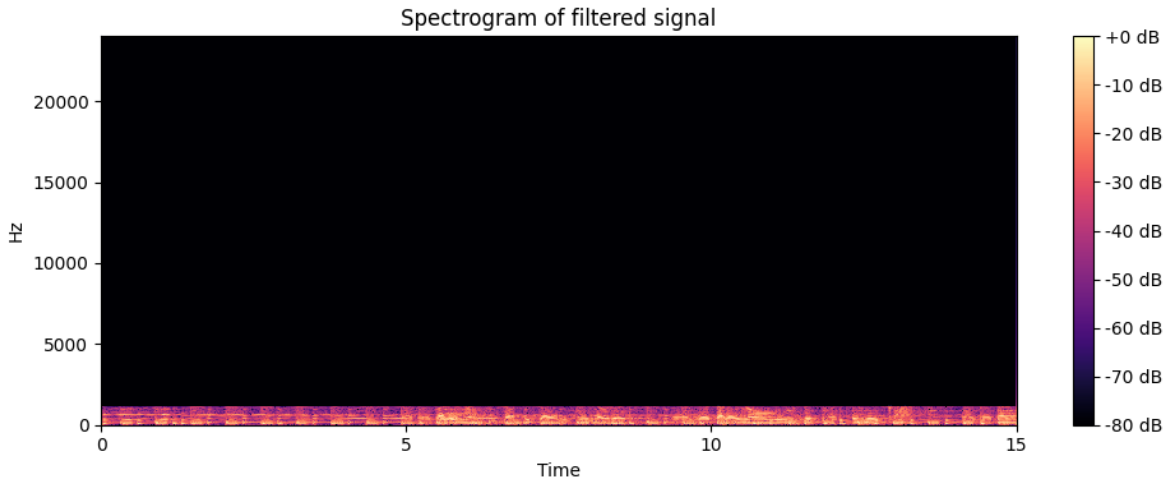

Figure 6: Spectrogram of Cleaned Audio

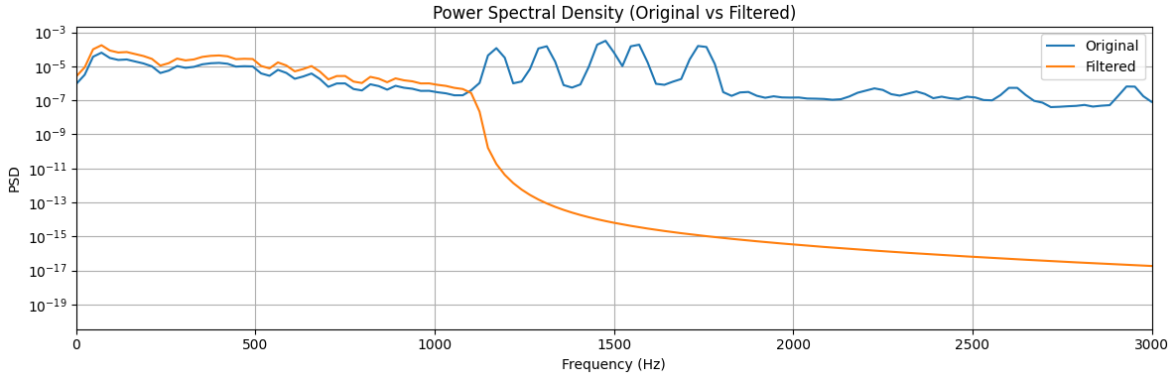

Figure 7: Spectral Power Density of Cleaned Audio

9 Conclusion

We can clearly see that in the final cleaned audio, only the indie music is heard and the unwanted piccolo music is removed. Attached with the report is the cleaned audio file. As a fun end to the assignment, the cleaned audio was played to Google Search Music feature, and it identified the music as 'Everybody Knows' by Leonard Cohen, a synth-pop indie song recorded in 1988!